

Úvod do databázových systémů

Kapitola 3
Konceptuální modelování

Konceptuální modelování

- konceptuální model
 - popis dat v DB nezávisle na fyzické a logické implementaci
 - výsledek je vstupem do dalších fází návrhu DB
 - databázi lze na jeho základě jednoznačně realizovat, přitom nás ale nezavazuje ke konkrétní platformě
 - v konceptuálním modelu se staráme hlavně o popis reality, ne o řešení v konkrétním systému
 - má co nejvěrněji vystihnout lidský pohled
 - aby se analytik a budoucí uživatel pochopili
 - definuje **konceptuální schéma** databáze

Konceptuální modelování

- co když konceptuální model neuděláme
 - příliš nízká úroveň pohledu na data
 - obtížná komunikace zadavatele a analytika
 - nemožnost realizace větší DB (obtížná orientace)
- vztah konceptuálního modelu a konkrétních dat
 - konceptuální model popisuje **obecnou strukturu** dat, která chceme do DB ukládat
 - tvorba konceptuálního modelu vychází ze znalosti toho, jak to v modelované skutečnosti „chodí“
 - snaha o zobecnění a zachycení podstatných jevů
 - model musí být navržen tak, aby „pasoval“ na všechny situace, které potřebujeme databází podchytit

Konceptuální modelování

- obecné prvky každého konceptuálního modelu
 - entita (třída)
 - entita je kategorie objektů, jež jsou z nějakého hlediska „stejné“
 - vztah (asociace)
 - zobecněná závislost mezi entitami
 - integritní omezení
 - podmínky, které je třeba hlídat, aby databáze byla korektní (odpovídala realitě)
 - nutno ještě dodefinovat takové, které nejsou zachyceny na úrovni entit a vztahů

Konceptuální modelování

■ entita (třída)

- musí být navržena tak, aby dokázala pojmut konkrétní datové prvky
 - tj. má instance (objekty)
- popis entity (třídy) je dán především popisem jejích jednotlivých atributů
 - název, doména, povinnost, identifikace, unikátnost...

Konceptuální modelování

■ **vztah (asociace)**

- musí být navržen tak, aby dokázal pojmut konkrétní vztahy mezi datovými prvky
- různé konceptuální modely mají různé typy vztahů
 - obecný vztah (asociace) – kardinalita, parcialita
 - identifikační vztah (slabá a silná entita)
 - ISA-hierarchie (typ a podtyp jako specializace)
 - pozor na sémantiku, opatrně a nepřehánět
 - ...

Konceptuální modelování

■ **entity a vztahy**

- musí být navrženy na optimální úrovni detailu
 - nesmí být ani zbytečně obecné
 - pak by do nich šlo uložit cokoli a databáze by byla nepřehledná a špatně strukturovaná
 - nesmí být ani málo obecné (příliš speciální)
 - pak by došlo k zbytečnému rozdrobení databáze na mnoho vzájemně podobných struktur
 - nesmějí se svou působností překrývat
 - (až na výjimky)
 - musí být vždy jasné, kam konkrétní datový prvek či vztah patří

Konceptuální modelování

■ integritní omezení (IO)

- podmínky (tvrzení), vymezující korektnost DB
 - libovolná omezení na úrovni objektů, jejich vztahů, konkrétních dat...
 - např. každá osoba musí být buď student nebo učitel, ale ne obojí
 - každý předmět musí mít právě jednoho garanta
 - student si smí zapsat nejvýše 6 předmětů naráz
 - aby se student mohl přihlásit na zkoušku, musí mít zápočet
 - jméno osoby nesmí obsahovat číslici
 - semestr může být pouze letní nebo zimní
 - identifikátorem předmětu je jeho kód
 - název předmětu nesmí být prázdný
 - ...

Konceptuální modelování

■ integritní omezení (IO)

- vyplývají z povahy modelované skutečnosti, tj. jejich existenci nemůžeme nijak ovlivnit
- musíme je co nejpřesněji identifikovat
 - definujeme je již na konceptuální úrovni
 - na logické úrovni mohou přibýt další IO, dané použitým log. modelem
 - např. volba cizího klíče, způsob ošetření povinného členství ve vztahu apod.
- v databázi se pak definují pomocí jazyka pro definici dat (DDL), ale ten někdy na ošetření všech IO nestačí
 - nutnost dalších prostředků – triggerů, uložené procedury, ošetření v aplikaci

Jak na to...

- klasický přístup – strukturovaná metodika
 - oddělená funkční a datová analýza
 - **datové schema** – výsledek datové analýzy a návrhu
 - jaká budou v systému data, jaké mají vztahy
 - **funkční schema** – výsledek funkční analýzy a návrhu
 - co bude systém dělat (jaké funkce poskytuje)
 - pracovní postupy organizace, které má systém počítačově podporovat
 - události, spouštějící počítačově podporovaný pracovní postup
 - kdo bude systém používat
 - problém: udržení vzájemného souladu (konzistence) funkční a datové analýzy

Jak na to...

- objektová metodika
 - funkční a datová analýza není striktně oddělená
 - zdůrazňuje zapouzdření dat
 - pracuje s voláním metod na objektech
 - masivně využívá konstrukce jako agregace, kompozice, dědění
 - UML notace je objektově orientovaná

Jak na to...

- nejčastější konceptuální modely

- UML Class Diagram

- objektový, data modelována třídami a asociacemi
- výhodný pro OO databáze, jenže většina DB je stejně relačních

- entitně-vztahový (ER model, entity-relationship)

- datové prvky (entity) a vztahy mezi nimi
- standard pro modelování relačních DB

- pro návrh databáze se často nevyužívají všechny výrazové prostředky

- jednodušší diagram má přímočarý překlad do struktur zvoleného logického modelu, převod složitějších konstrukcí nemusí být jednoznačný nebo úplný a může působit problémy, viz např. ISA

Jak na to...

- při modelování projektant na základě podrobného seznámení s modelovanou realitou
 1. identifikuje **typy entit** (třídy) jako kategorie objektů téhož typu, například „SILNICE“, „MOST“ atd.
 2. identifikuje **typy vztahů**, do nichž mohou entity vstupovat, např. MOST (entita) *LEŽÍ NA* (vztah) SILNICI (entita)
 3. přiřadí typům entit a typům vztahů vhodné **atributy**, které popisují jejich vlastnosti
 4. formuluje **integritní omezení**, vyjadřující soulad schématu s modelovanou realitou, např. MOST musí ležet na nějaké SILNICI; SILNICE je identifikována číslem

ER model

- množina pojmů, umožňujících popis struktury DB
 - uživatelsky srozumitelně, ale zároveň přesně a jednoznačně
 - nejlépe se vyjadřuje graficky
- základní konstrukty
 - entita
 - vztah
 - atribut
- další rozšiřující a speciální konstrukty
 - slabá entita, rekurzivní vztah, ISA hierarchie, výlučný vztah, speciální typy atributů, ...

ER model

- nejednoznačnost v terminologii
 - jako označení abstraktního objektu by měl být používán pojem „*typ entity*“: analogie třídy v OOP
 - pojem „*entita*“ by měl být použit pro označení konkrétního výskytu objektu: analogie instance třídy v OOP
 - podobně je někdy třeba rozlišovat mezi „*typem vztahu*“ a (konkrétním) „*vztahem*“

ER model

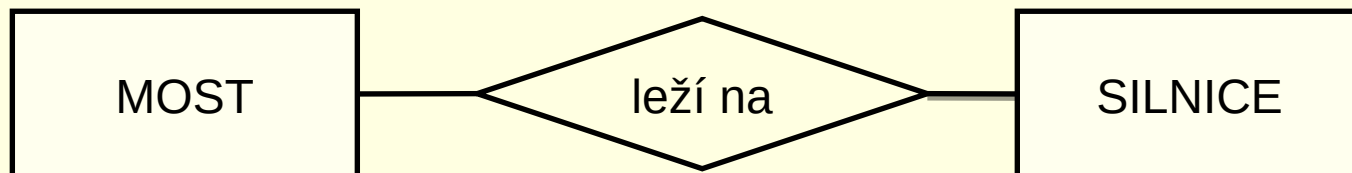
■ **entitní typ (entity)**

- popisuje kategorii objektů reálného světa
 - je obvykle odpovědí na otázku „co“, je vystižena podstatným jménem
 - každou takovou „kategorii“ dobře vymezit vůči ostatním kategoriím v modelu – entity se neprolínají
 - objekty, patřící do dané entity, mají tutéž strukturu („vypadají analogicky“), ale každý z nich je schopen nezávislé existence a je jednoznačně identifikovatelný
- např. pro evidenci mostů zavedeme entitu MOST
 - do ní budou náležet všechny evidované mostní objekty, třeba most č. 221-0608 a most č. 58-204c

MOST

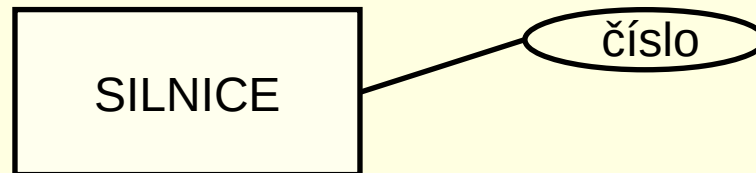
ER model

- **vztahový typ** (relationship)
 - vazba mezi dvěma (či více) entitami
 - často je vztahový typ vystižen slovesem
 - na úrovni koncept. modelu jde o obecnou vazbu mezi entitami, nikoli o vazbu mezi konkrétními objekty – konkrétní vztahy jsou ve vztahovém typu obsaženy
 - např. víme, že mosty leží na silnících, tj. entita „most“ je ve vztahu „leží na“ s entitou „silnice“
 - konkrétně pak třeba most č. 221-0608 leží na silnici třetí třídy č. 221



ER model

- atribut (attribute)
 - nějaká popisná hodnota, přiřazená entitě či vztahu; např. entita „SILNICE“ má atribut „číslo“

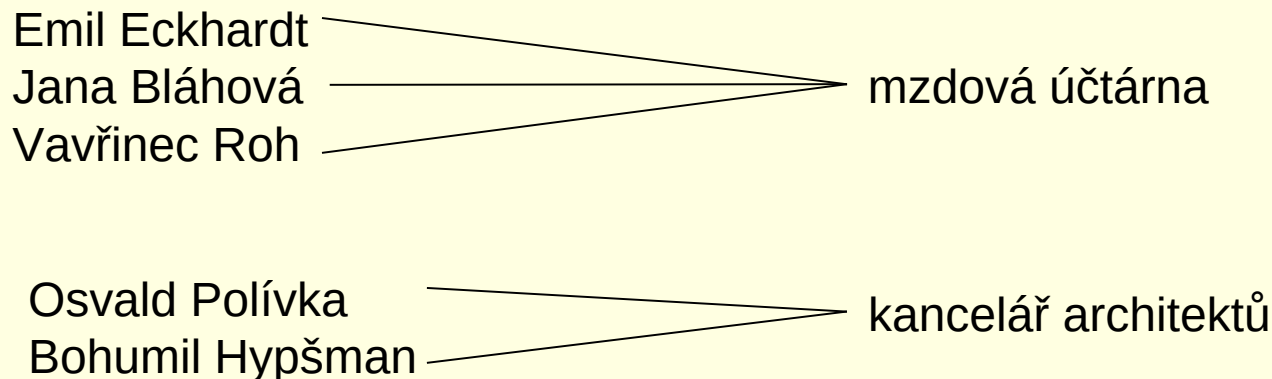


Typ entity, typ vztahu

- „zaměstnanec Emil Eckhardt, rč. 105516246“ je konkrétní osoba, charakterizovaná množinou vlastností, které nás zajímají z hlediska zaměstnavatele
- osob s touto rolí může být evidentně více, říkáme, že pan Eckhardt je entitou typu ZAMĚSTNANEC
- každý zaměstnanec je zaměstnán na právě jednom oddělení, např. pan Eckhardt pracuje ve mzdové účtárně – potřebujeme tedy také entitní typ ODDĚLENÍ a vztahový typ *PRACUJE NA*

Typ entity, typ vztahu

■ diagram výskytů



ZAMĚSTNANEC

PRACUJE NA

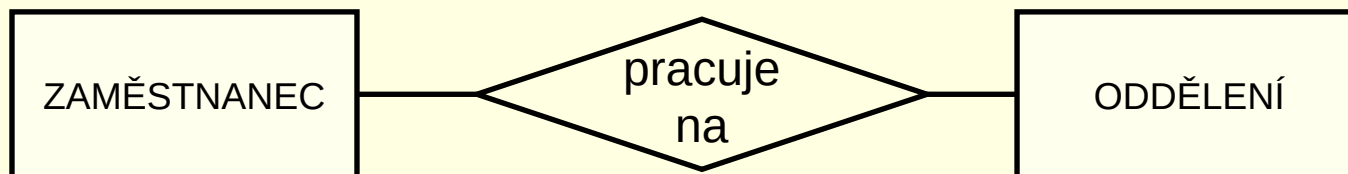
ODDĚLENÍ

■ dva entitní a jeden vztahový typ

- pět výskytů ent. typu ZAMĚSTNANEC
- dva výskyty ent. typu ODDĚLENÍ
- pět výskytů vztahového typu *PRACUJE NA*

Typ entity, typ vztahu

- diagram typů entit a vztahů



Identifikační klíč

- každá entita musí být jednoznačně identifikovatelná
- atribut nebo skupina atributů, jejichž hodnota slouží k identifikaci konkrétní entity se nazývá **identifikační klíč** (v E-R schematu podtrženě)
 - např. atribut „rodné číslo“ entity ZAMĚSTNANEC
- entitní typ může mít několik kandidátů na klíč
 - ZAMĚSTNANEC může být identifikován
 - rodným číslem
 - číslem zaměstnance
 - jménem, příjmením a datem narození

Identifikační klíč

- volba identifikačního klíče
 - klíč může být buď přirozený (jeden z atributů entity) nebo umělý (dodatečný atribut „id“)
 - pokud entita obsahuje vhodný, nejlépe číselný atribut, pak jej lze použít jako klíč
 - pokud by klíčem entity měla být větší skupina atributů nebo např. delší textový atribut, pak je efektivnější zavést klíč umělý
 - složené klíče činí potíže při indexování, při navazování referenční integrity (viz níže), apod...

Identifikační klíč

- volba identifikačního klíče
 - je tedy umělý identifikátor vždy ideální řešení???
 - není:
 - nedokáže zajistit, aby se jeden a tentýž objekt nevyskytnul v databázi dvakrát: budou-li mít dvě instance entity zcela totožné hodnoty všech atributů a lišit se budou pouze hodnotou umělého identifikátoru, zjevně jde o dva stejné objekty zadané dvakrát, pouze pod jiným číslem – téměř ekvivalentní stavu, kdy by entitní typ neměl vůbec žádný identifikační klíč – dvě či více instancí entity pak může mít stejné hodnoty všech atributů

Identifikační klíč

- volba identifikačního klíče
 - je tedy umělý identifikátor vždy ideální řešení???
 - není:
 - z hlediska databázové teorie, zejména relačního modelu (viz další přednášky), je výše uvedená situace nežádoucí, neboť eliminuje některé jinak velmi užitečné principy klíče (omezení redundance, omezení nekonzistence, jednoznačnost entit)
 - je třeba nalézt kompromis a nevýhody použití umělého identifikátoru omezit např. preciznější definicí a kontrolou integritních omezení, včetně uložených procedur, triggerů, a též na úrovni aplikační vrstvy

Přiřazení atributů

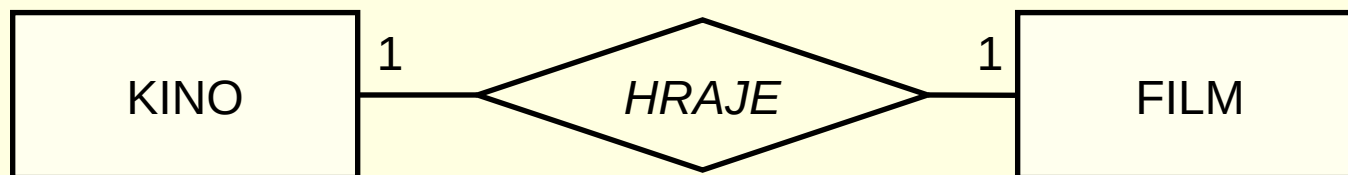
- typy entit a vztahů mají přiřazeny atributy, které je charakterizují
- v klasickém pojetí jsou jednotlivé atributy atomické, tj. dále nedělitelné (tzv. 1. normální forma)
- atributy také podléhají **integritním omezením**
 - **datový typ** (množina hodnot, množina operací, paměťový prostor...)
 - příznak, zda je atribut **klíčový**
 - příznak, zda atribut může být prázdný (**NULL**)
 - příznak, že atribut musí mít **unikátní** hodnotu
 - zjevně klíčový atribut musí být UNIQUE a NOT NULL

Kardinalita (poměr) vztahu

- **integritní omezení** pro vztahový typ
- entitní typy KINO a FILM a vztahový typ *HRAJE*
- tři druhy kardinality
 - poměr 1:1
 - poměr 1:N
 - poměr M:N

Kardinalita (poměr) vztahu

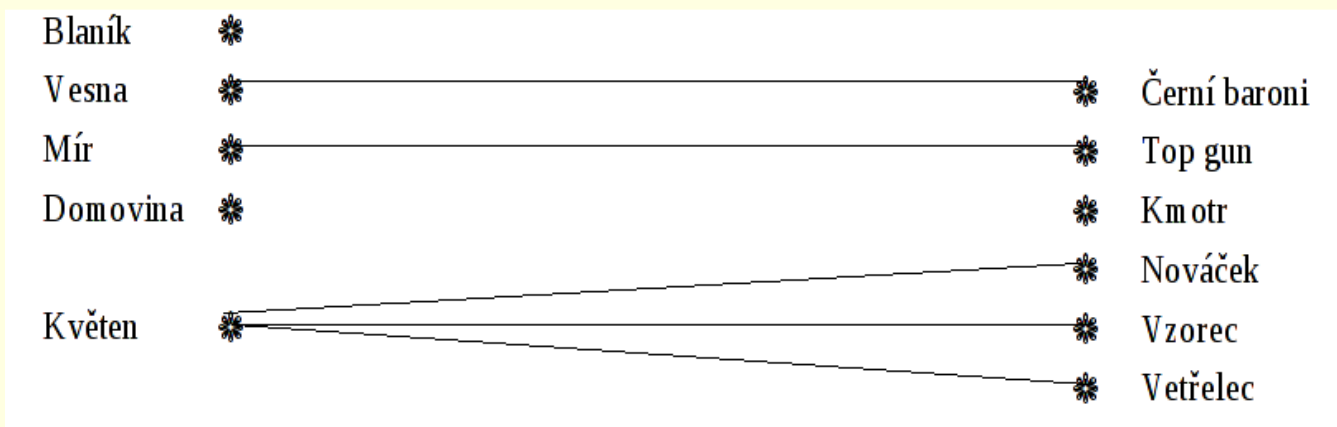
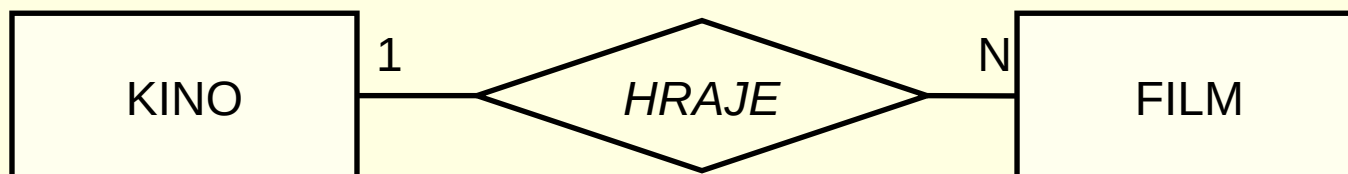
- poměr 1:1
 - dané kino dává maximálně jeden film
 - daný film je na programu maximálně v jednom kině
 - může zahrnovat vztahy 0:1 a 1:0, tj. kino nic nedává nebo film se nikde nehraje – povinnost členství



Blaník	✱		
Vesna	✱	—————	✱ Černí baroni
Mír	✱	—————	✱ Top gun
Domovina	✱	—————	✱ Kmotr
			✱ Nováček
Květen	✱		✱ Vzorec
			✱ Větrlec

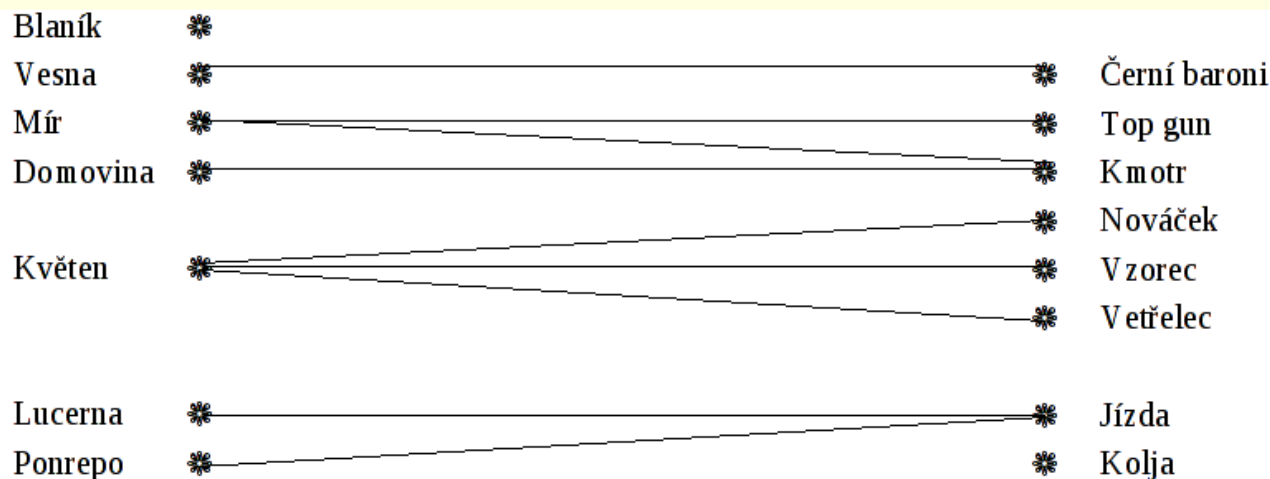
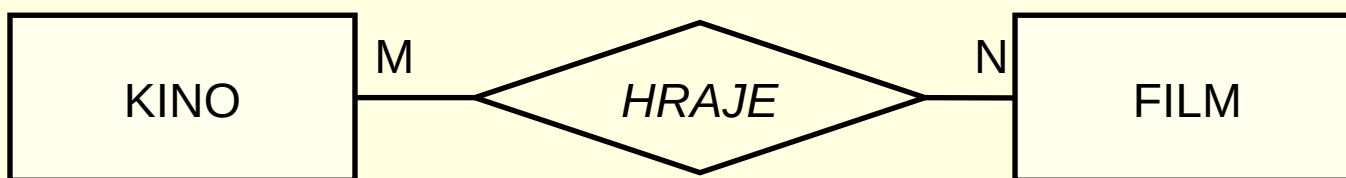
Kardinalita (poměr) vztahu

- poměr 1:N
 - dané kino může hrát více než jeden film
 - daný film se dává maximálně v jednom kině
 - může zahrnovat vztahy 0:1, 1:0, 1:1
 - důležitý směr („jedno kino – víc filmů“ vs. „jeden film – víc kin“)



Kardinalita (poměr) vztahu

- poměr M:N
 - dané kino může hrát více než jeden film
 - daný film se může dávat ve více než v jednom kině
 - může zahrnovat vztahy 0:1, 1:0, 1:1, 1:N, N:1 – některé mohou být vyloučeny přísnějšími pravidly



Kardinalita (poměr) vztahu

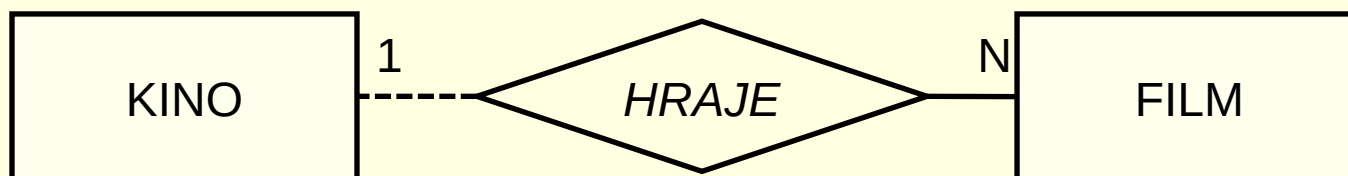
- kardinalita také odpovídá tvrzení, že jedna entita jednoznačně určuje druhou entitu, resp. je **determinantem** entity druhého typu
 - pro vztah 1:1 lze např. říct, že název kina determinuje název filmu a také název filmu determinuje název kina
 - pro vztah 1:N lze např. říct, že název filmu determinuje název kina, ale název kina nedeterminuje název filmu
 - u vztahu M:N není determinující ani jedna entita

Parcialita (členství) vztahu

- účastní-li se entita vztahu, říkáme, že je členem vztahu
- někdy má entita předepsanou účast ve vztahu, tj. členství je povinné
 - např. zaměstnanec musí být zaměstnán na nějakém oddělení
- jindy se entita do vztahu zapojovat nemusí, členství je nepovinné
 - např. oddělení může existovat i bez zaměstnanců
- důležité: vyjadřuje, že entita nemůže existovat bez zapojení do vztahu s instancí druhého typu

Parcialita (členství) vztahu

- v grafickém konceptuálním modelu se parcialita značí různě, například



- v kině se **může** dávat víc filmů anebo žádný
- film se **musí** dávat právě v jednom kině

Integritní omezení pro vztahy, souhrn

- kardinalita a parcialita jsou důležitá IO
 - na jejich volbě závisí efektivita implementace DB
 - jejich nastavením je dáno, jaká data půjde či nepůjde vložit do databáze!
 - je-li někde nastaveno povinné členství dat. prvku X, pak musí v DB vždy existovat k X odpovídající partner, jinak prvek X do databáze vůbec nevložím
 - jak potom inicializovat databázi??
- ALE: čím přesněji je databázové schema definováno při návrhu, tím méně chyb se může vloudit na úrovni aplikačních programů
 - není tedy vhodné zjednodušovat si práci na návrhu přílišnou benevolencí (např. nastavovat všem entitám nepovinná členství)

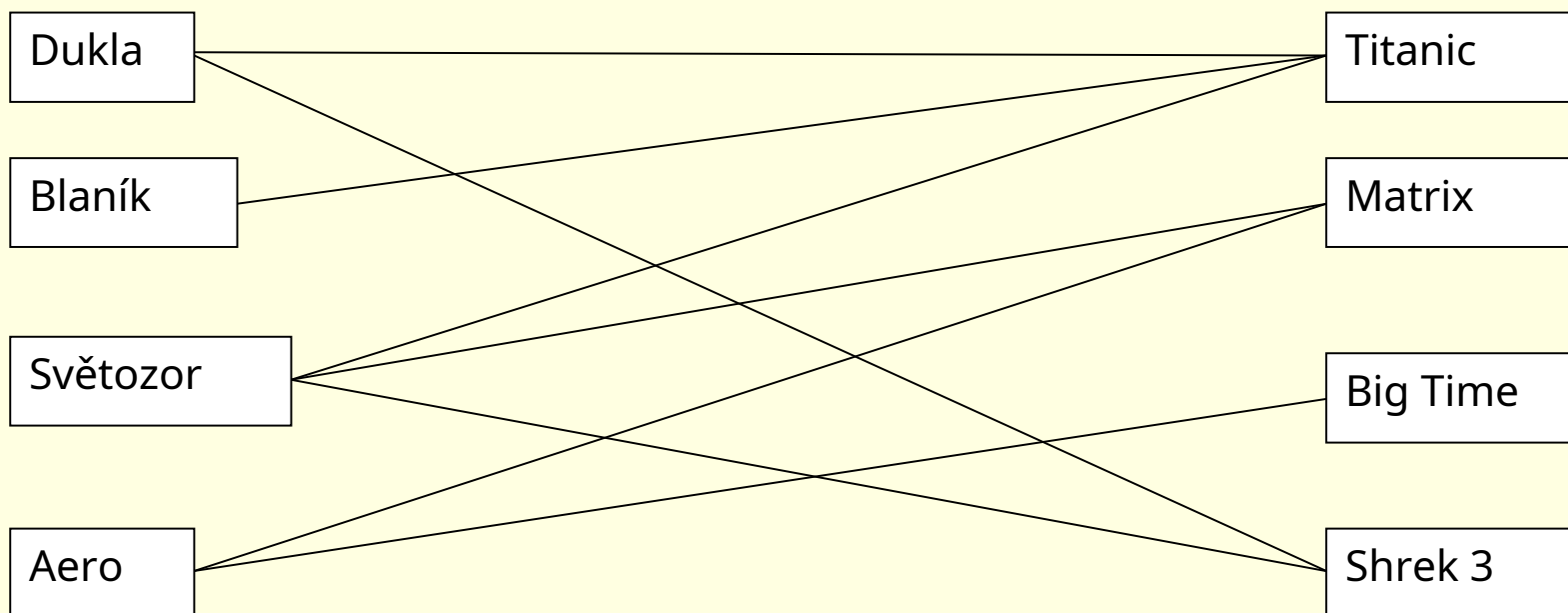
Příklad

■ zadání

- mějme seznam kin a seznam filmů, v každém kině se může dávat víc filmů a každý film se může dávat ve více kinech
 - kino má atributy: název, adresa, počet míst, Dolby
 - film má atributy: název, rok, země vzniku, dabing
- navrhněme E-R model ve dvou variantách
 - A: chceme evidovat jen filmy, které jsou někde dávány
 - B: chceme evidovat všechny filmy

Příklad

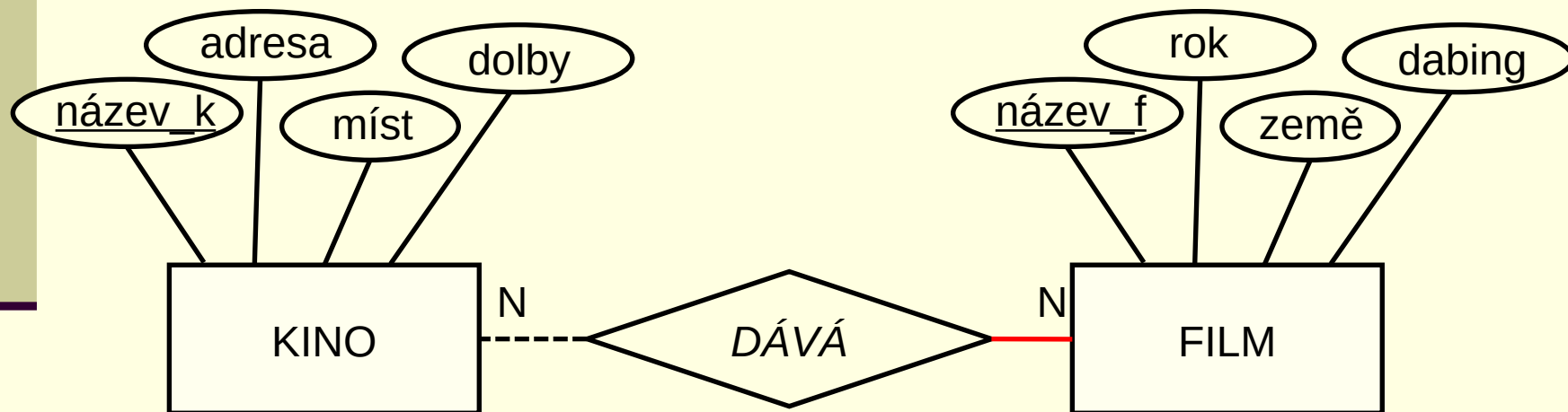
■ příklad výskytového diagramu



Příklad

■ řešení

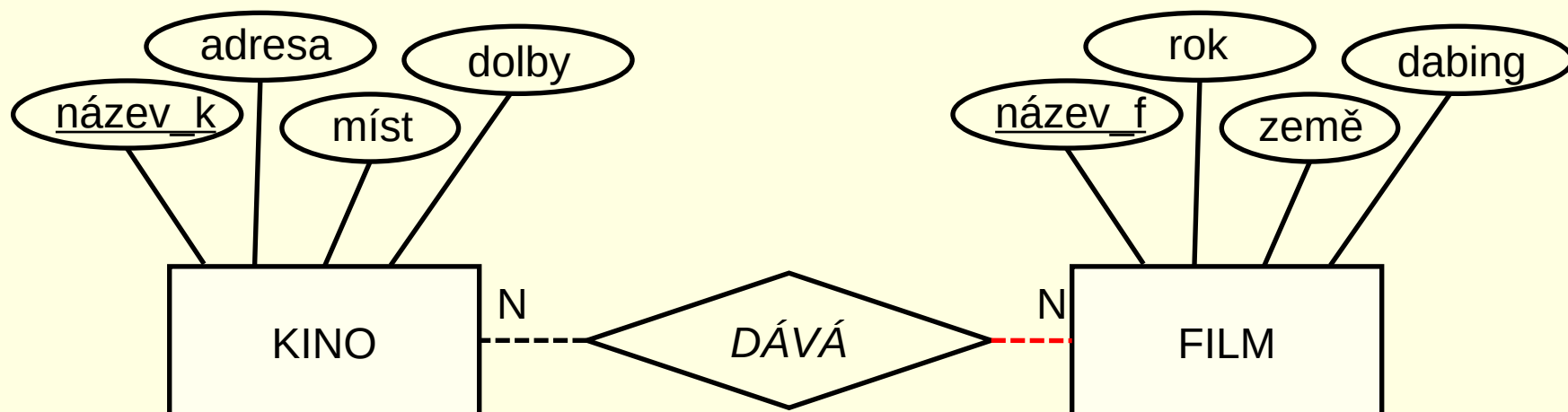
- entity: Kino, Film
- vztahy: Dává (M:N) mezi Kinem a Filmem
 - A: chceme evidovat jen filmy na programu
 - povinné členství filmu ve vztahu



Příklad

■ řešení

- B: chceme evidovat libovolné filmy
 - nepovinné členství filmu ve vztahu



- ??? – co kino, má mít povinné nebo nepovinné členství ve vztahu?
 - není zadáním řešeno, nutno se zeptat zadavatele...